SOKOINE UNIVERSITY OF AGRICULTURE

School of Engineering and Technology
Department of Agricultural Engineering

AGE 219: Basics of Computer Programming

Lecture Compendium 8: Introduction to Object-Oriented Programming (OOP) — Modeling Agricultural Systems as Digital Entities

---

Prepared by:

Dr. Kadeghe Fue, PhD, P.Eng (T)

Target Audience: Engineering Students (Agricultural, Irrigation, Post-Harvest, and related fields)
Primary Tools: Python 3.x and Google Colaboratory
Location: Morogoro, Tanzania

This lecture marks a fundamental shift in how we structure code. Instead of viewing a program as a list of instructions, we will view it as a collection of interacting objects. This "Object-Oriented"

paradigm is the standard for building complex agricultural simulations, robotics control systems, and management software.

Contents

1 Introduction: The Engineering Blueprints of Code **2**
1.1 The Analogy: Class vs. Object 2

2 Defining a Class: The Attributes and Behaviors **2**
2.1 Basic Syntax 2

3 The Constructor: `__init__` and `self` **2**
3.1 The Mystery of `self` 3

4 Methods: Defining Behaviors **3**

5 Encapsulation: Protecting Engineering Data **3**

6 Case Study 1: Modeling an Intelligent Soil Sensor Node **4**

7 Case Study 2: The Virtual Tractor Simulator **4**

8 Introduction to Inheritance: Specializing Machines **5**

9 Why OOP is Crucial for Precision Agriculture **5**

10 Summary **6**

11 Glossary of Terms **7**

12 Extended Self-Assessment Exercises **8**
12.1 Exercise 1: The Grain Silo Manager 8
12.2 Exercise 2: Veterinary Herd Tracker 8
12.3 Exercise 3: Advanced Inheritance 8
12.4 Exercise 4: The Debugging Challenge 8

1 Introduction: The Engineering Blueprints of Code

In our journey through AGE 219, we have progressed from basic variables to complex data structures and reusable functions. This style of programming is called **Procedural Programming**. It works well for linear tasks, such as reading a sensor and calculating an average.

However, as agricultural engineering projects scale, procedural code becomes difficult to manage. If you are developing a "Digital Twin" of a 100-node irrigation network, managing every pump, valve, and sensor as a disconnected variable or function leads to "spaghetti code."

Object-Oriented Programming (OOP) solves this by allowing us to group data and behavior into single, cohesive entities called **Objects**.

1.1 The Analogy: Class vs. Object

Think of a **Class** as an engineering drawing or a **Blueprint**. It describes the specifications of a machine (e.g., a tractor) but it is not the machine itself. An **Object** is the physical machine built from that blueprint. You can build 50 different tractors from one blueprint; each tractor is an **Instance** of the class.

- **Class:** The "Tractor" Blueprint.
- **Object:** The specific Massey Ferguson 4700 parked in the SUA workshop.

2 Defining a Class: The Attributes and Behaviors

In OOP, we describe entities using two components:

1. **Attributes (Data):** What the object *is* or *has* (e.g., horsepower, fuel level, brand).
2. **Methods (Behavior):** What the object *does* (e.g., move forward, engage PTO, consume fuel).

2.1 Basic Syntax

We use the `class` keyword. Note that class names in Python traditionally use **PascalCase** (e.g., `TractorEngine`, not `tractor_engine`).

```
class SimpleSensor:
    # A placeholder for now
    pass

# Creating an 'instance' (object) of the class
my_sensor = SimpleSensor()
```

3 The Constructor: `__init__` and `self`

The most important method in a Python class is `__init__`. This is the "Constructor." It is automatically called when you create a new object. It initializes the object's attributes.

3.1 The Mystery of `self`

In every method within a class, the first parameter is always `self`. **Self** represents the specific instance of the object being created. It allows the code to distinguish between ”Tractor A’s fuel” and ”Tractor B’s fuel.”

```
class Tractor:
    def __init__(self, brand, hp):
        # Assigning values to 'self' attributes
        self.brand = brand
        self.horsepower = hp
        self.fuel_level = 100.0 # Standard starting fuel
        print(f"New {brand} tractor initialized with {hp} HP.")

# Creating two different tractor objects
tractor_a = Tractor("John Deere", 120)
tractor_b = Tractor("Massey Ferguson", 85)

print(tractor_a.brand) # Output: John Deere
print(tractor_b.brand) # Output: Massey Ferguson
```

4 Methods: Defining Behaviors

Methods are functions defined inside a class. They operate on the data stored within that specific object.

```
class IrrigationPump:
    def __init__(self, model, max_flow_lps):
        self.model = model
        self.max_flow = max_flow_lps
        self.is_running = False

    def start_pump(self):
        self.is_running = True
        print(f"Pump {self.model} is now ACTIVE.")

    def stop_pump(self):
        self.is_running = False
        print(f"Pump {self.model} is now SHUT DOWN.")

# Usage
my_pump = IrrigationPump("SUA-Hydra-200", 15.5)
my_pump.start_pump()
```

5 Encapsulation: Protecting Engineering Data

Encapsulation is the practice of hiding the internal details of an object and only exposing what is necessary. In engineering, this prevents someone from accidentally changing a critical calibration constant.

In Python, we signal that an attribute should be ”private” by starting its name with an underscore (_).

```
class SoilProbe:
    def __init__(self, calibration_offset):
        # This is intended to be a private internal variable
        self._offset = calibration_offset

    def get_reading(self, raw_voltage):
```

```
        return (raw_voltage * 20) + self._offset

probe = SoilProbe(2.5)
# Users should call get_reading(), not modify ._offset directly
```

6 Case Study 1: Modeling an Intelligent Soil Sensor Node

Engineering Context: In a smart farm, sensors need to manage their own battery life and report when they need maintenance.

```
class SmartSensor:
    """Class representing an IoT Soil Moisture Node."""

    def __init__(self, sensor_id, block_name):
        self.id = sensor_id
        self.block = block_name
        self.battery_percent = 100.0
        self.readings_log = []

    def take_reading(self, moisture_value):
        if self.battery_percent < 5.0:
            print(f"ERROR: Sensor {self.id} battery too low to read.")
            return None

        self.readings_log.append(moisture_value)
        self.battery_percent -= 0.5 # Each reading consumes power
        return moisture_value

    def get_average(self):
        if not self.readings_log:
            return 0
        return sum(self.readings_log) / len(self.readings_log)

# Practical Implementation
node_1 = SmartSensor("IOT-001", "North Maize Plot")
node_1.take_reading(22.5)
node_1.take_reading(24.1)

print(f"Node {node_1.id} Average: {node_1.get_average()}%")
print(f"Battery remaining: {node_1.battery_percent}%")
```

7 Case Study 2: The Virtual Tractor Simulator

Engineering Context: We need to simulate how much fuel a tractor consumes based on the distance traveled and whether an implement is engaged.

```
class SimulatorTractor:
    def __init__(self, model, fuel_capacity):
        self.model = model
        self.fuel_tank = fuel_capacity # Liters
        self.current_fuel = fuel_capacity
        self.odometer = 0.0 # km
        self.is_plowing = False

    def toggle_plow(self):
        self.is_plowing = not self.is_plowing
        status = "ENGAGED" if self.is_plowing else "DISENGAGED"
        print(f"Implement is now {status}")

```

```
    def drive(self, distance_km):
        # Base consumption: 0.5 L/km
        # Plowing consumption adds 1.5 L/km
        rate = 2.0 if self.is_plowing else 0.5
        total_needed = distance_km * rate

        if self.current_fuel >= total_needed:
            self.current_fuel -= total_needed
            self.odometer += distance_km
            print(f"Drove {distance_km}km. Fuel left: {self.current_fuel:.1f}L")
        else:
            print("ALERT: Insufficient fuel for this distance!")

# Simulation Run
tractor = SimulatorTractor("MF-SUA", 200)
tractor.drive(10) # Normal driving
tractor.toggle_plow()
tractor.drive(50) # Heavy workload
```

8 Introduction to Inheritance: Specializing Machines

Inheritance allows a class to derive attributes and methods from another class. This is perfect for engineering hierarchies. For example, all **Machinery** has a Serial Number and Model, but a **Harvester** has grain-specific features that a **Tractor** does not.

```
class BaseMachine:
    def __init__(self, serial_no, manufacturer):
        self.sn = serial_no
        self.maker = manufacturer

class CombineHarvester(BaseMachine):
    def __init__(self, serial_no, manufacturer, hopper_capacity):
        # Use super() to call the parent class constructor
        super().__init__(serial_no, manufacturer)
        self.capacity = hopper_capacity
        self.grain_collected = 0

    def harvest(self, amount):
        self.grain_collected += amount
        print(f"Collected {amount}kg. Total: {self.grain_collected}/{self.capacity}")

# Creation
h1 = CombineHarvester("CH-2025-X", "Claas", 8000)
print(f"Machine SN: {h1.sn} made by {h1.maker}")
h1.harvest(500)
```

9 Why OOP is Crucial for Precision Agriculture

1. **Spatial Modeling:** Each grid cell in a yield map can be an object with properties like Nitrogen level, pH, and elevation. 2. **Fleet Management:** Real-time tracking of 100 harvesters is easy when each one is an instance of a `Harvester` class reporting its own coordinates. 3. **Simulation:** Testing a new irrigation logic is safer in a digital world where `Crop` and `Environment` are objects interacting via code.

10 Summary

In this lecture, we have transitioned from code-as-text to code-as-architecture. You have learned:

- The distinction between a **Class** (blueprint) and an **Object** (instance).
- How to use `__init__` to define the state of an engineering system.
- The role of `self` in managing multiple machines simultaneously.
- How **Methods** define the physical actions of a machine.
- The power of **Inheritance** for creating specialized agricultural tools.

In Lecture 9, we will combine these structures with powerful libraries like **Pandas** and **Matplotlib** to analyze and visualize the data generated by our objects.

11 Glossary of Terms

- **Attribute:** A variable that belongs to an object, representing its state.
- **Class:** A blueprint for creating objects, defining shared attributes and methods.
- **Constructor (`__init__`):** A special method that initializes an object's state upon creation.
- **Encapsulation:** The grouping of data and methods into a single unit, often hiding internal implementation details.
- **Inheritance:** The process where one class acquires the properties and behaviors of another.
- **Instance:** A specific realization of any object (e.g., 'tractor_1' is an instance of 'Tractor').
- **Method:** A function defined inside a class that describes the behavior of an object.
- **Object:** A self-contained entity that consists of both data and procedures to manipulate that data.
- **Self:** A reference to the current instance of the class, used to access variables that belong to the class.

12 Extended Self-Assessment Exercises

12.1 Exercise 1: The Grain Silo Manager

Create a class named `GrainSilo`.

- **Attributes:** `capacity` (float), `current_stock` (float), `crop_type` (string).
- **Method** `add_grain(amount)`**:** Adds grain to the silo. If it exceeds capacity, print an alert and do not add.
- **Method** `check_status()`**:** Returns the percentage of the silo that is full.

Task: Instantiate a silo for "Maize" with 500-ton capacity and add 120 tons. Print the status.

12.2 Exercise 2: Veterinary Herd Tracker

Create a class `Livestock`.

- **Attributes:** `animal_id`, `weight_kg`, `is_healthy` (boolean).
- **Method** `record_weight(new_weight)`**:** Updates the weight.
- **Method** `mark_sick()`**:** Changes the health status.

Task: Create a list of 3 `Livestock` objects. Use a loop to print the IDs of all animals that are sick.

12.3 Exercise 3: Advanced Inheritance

Create a base class `Building`. Create a subclass `Greenhouse` that inherits from it.

- `Building` should have attributes `area` and `location`.
- `Greenhouse` should add attributes `glass_type` and `has_automation` (boolean).

Task: Instantiate a greenhouse at "SUA Farm 1" and print all its inherited and new properties.

12.4 Exercise 4: The Debugging Challenge

Identify the error in the following class definition:

```
class Motor:
    def __init__(brand, power):
        self.brand = brand
        self.power = power

m1 = Motor("Siemens", 15)
```

End of Lecture 8 Compendium